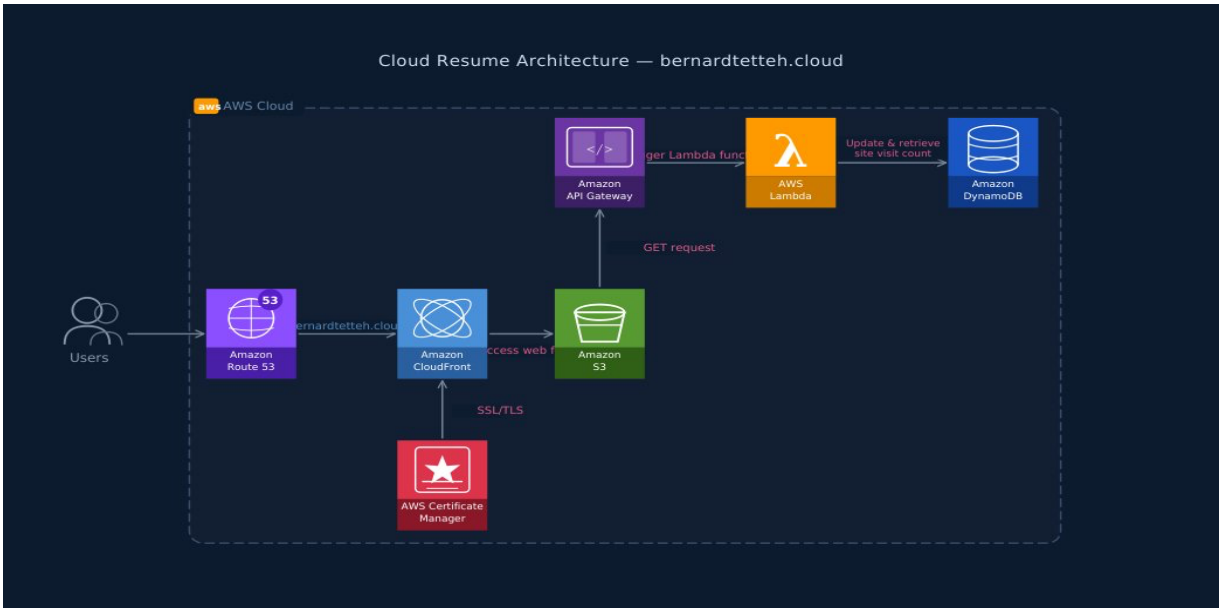
CLOUD RESUME CHALLENGE

bernardtetteh.cloud

A production grade, fully serverless resume website built on AWS. This document provides a detailed walkthrough of every AWS service used with real screenshots from the AWS console plus the real challenges encountered during the build and exactly how each one was solved.



Full architecture: Route 53 → CloudFront → S3 → API Gateway → Lambda → DynamoDB

7	**100%**	**~$0.50**	**6**
AWS Services	Serverless	Per Month	Challenges Solved

Bernard Tetteh · AWS Certified Solutions Architect · bernardtetteh.cloud · 2026

HOW IT ALL WORKS

Architecture Overview

When a visitor types bernardtetteh.cloud into their browser, seven AWS services work together automatically to serve the website and update the visitor counter all without a single traditional server.

#	Service	Role	What it does
1	**Route 53**	DNS	Translates domain → CloudFront IP address
2	**CloudFront**	CDN	Serves site globally over HTTPS from 400+ edge locations
3	**ACM**	Security	Free SSL/TLS certificate — gives site HTTPS padlock
4	**S3**	Storage	Stores all website files (HTML, CSS, JS)
5	**API Gateway**	API Layer	Receives GET /count request from browser JavaScript
6	**Lambda**	Compute	Python code that increments the visitor count atomically
7	**DynamoDB**	Database	Permanently stores the visitor count number

User → Route 53 → CloudFront + ACM → S3 → API Gateway → Lambda → DynamoDB

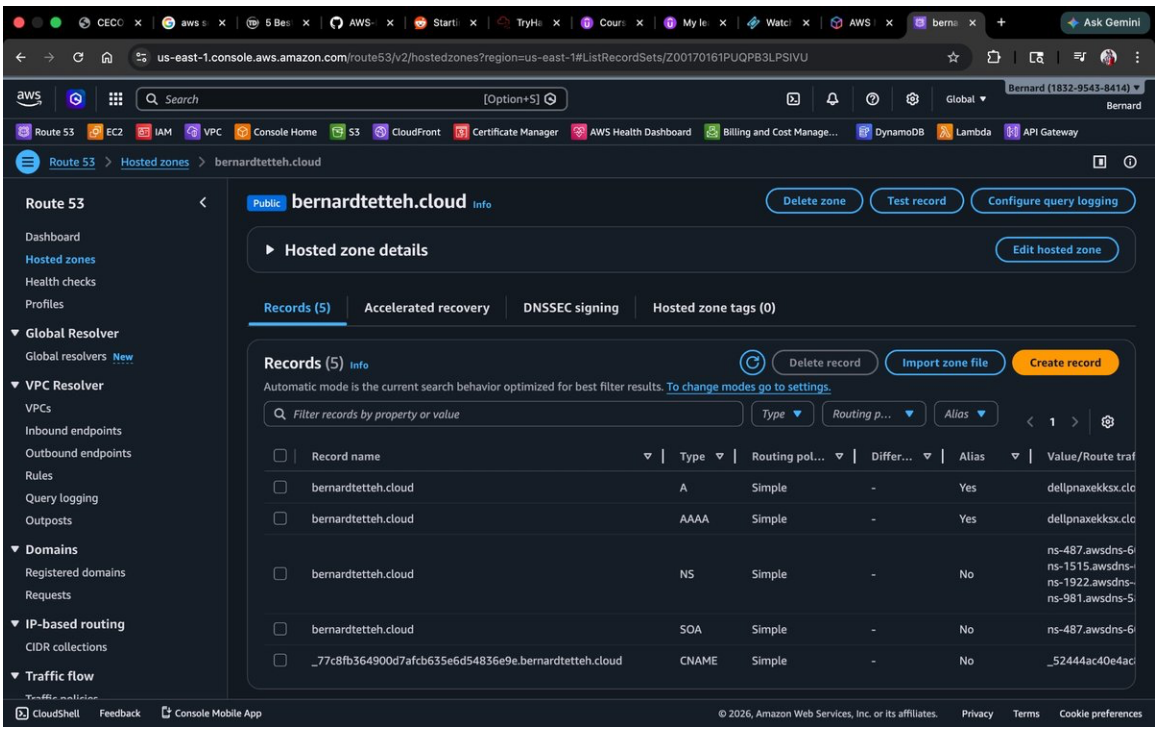
01 Amazon Route 53

Route 53 is where I bought, registered and manage my domain name bernardtetteh.cloud. It acts as the phonebook of the internet translating the human readable domain name into the IP address of my CloudFront distribution in milliseconds.

What I configured:

→ A record (Alias) — points bernardtetteh.cloud to the CloudFront distribution
→ AAAA record (Alias) — same as above for IPv6 visitors
→ CNAME record — added by ACM for SSL certificate DNS validation
→ NS + SOA records — auto-created by Route 53 when the hosted zone was set up

Setting	Value
Hosted Zone	bernardtetteh.cloud
Domain source	Registered directly via Route 53
Total records	5 (A, AAAA, NS, SOA, CNAME)
Routing	Simple routing policy
A record type	Alias → CloudFront distribution



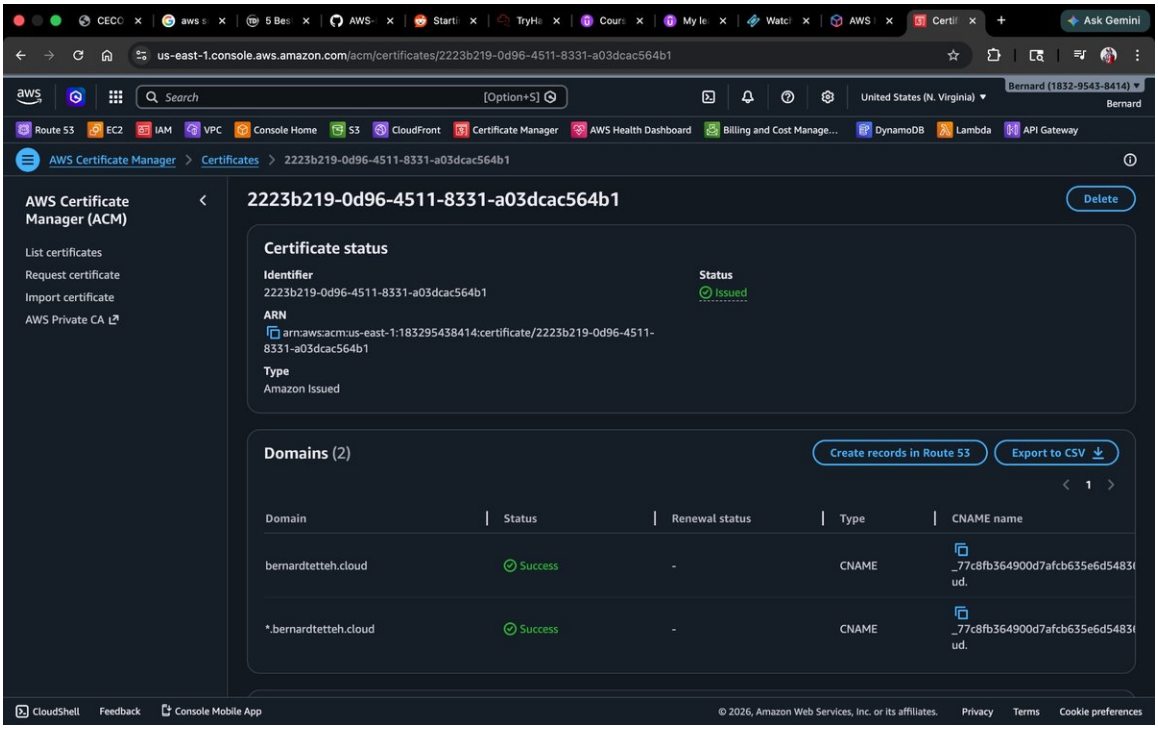
Route 53 hosted zone — bernardtetteh.cloud with all 5 DNS records. A and AAAA records alias to CloudFront.

02 AWS Certificate Manager (ACM)

ACM provisions a free SSL/TLS certificate enabling HTTPS the padlock in the browser. Without this, browsers would show a "Not Secure" warning. ACM auto-renews the certificate every 13 months with zero manual work. Important: ACM certificates for CloudFront must always be created in us-east-1 regardless of where other resources are.

→ Covers bernardtetteh.cloud and *.bernardtetteh.cloud (wildcard)
→ Validation: DNS method ACM adds a CNAME record to Route 53 automatically
→ Status: Issued both domains show Success
→ Cost: completely free, auto-renewing

Setting	Value
Status	Issued
Domains covered	bernardtetteh.cloud + *.bernardtetteh.cloud
Validation	DNS (CNAME in Route 53)
Region	us-east-1 (required for CloudFront)
Cost	Free



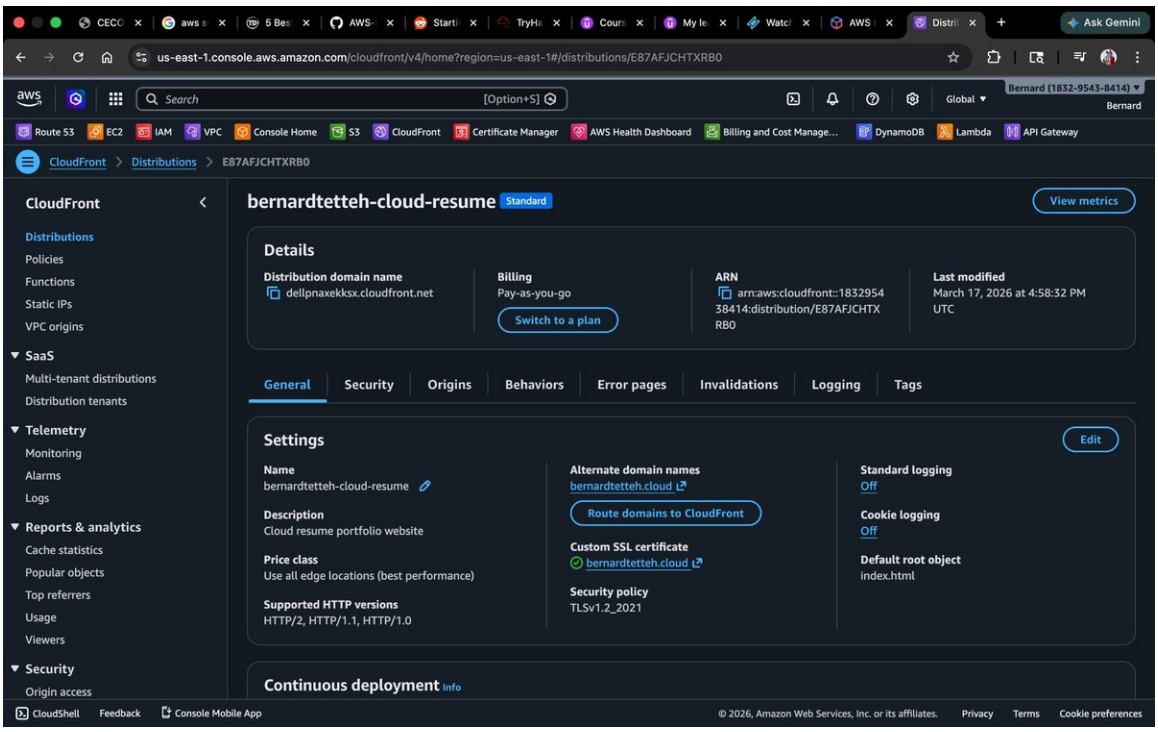
*ACM certificate Status: Issued. Both bernardtetteh.cloud and *.bernardtetteh.cloud validated successfully.*

03 Amazon CloudFront

CloudFront is Amazon's global Content Delivery Network (CDN). It caches the website at 400+ edge locations worldwide, so visitors always get the site from the nearest location fast, regardless of geography. CloudFront also handles HTTPS termination using the ACM certificate.

→ Distribution name: bernardtetteh-cloud-resume
→ Origin: S3 bucket (bernardtetteh.cloud)
→ Alternate domain: bernardtetteh.cloud — so my domain works instead of the cloudfront.net URL
→ Custom SSL certificate: attached from ACM
→ Default root object: index.html
→ Security policy: TLSv1.2_2021
→ Price class: All edge locations for best global performance

Setting	Value
Distribution name	bernardtetteh-cloud-resume
CloudFront domain	dellpnaxekksx.cloudfront.net
Alternate domain	bernardtetteh.cloud
SSL certificate	bernardtetteh.cloud (from ACM)
Default root	index.html
Security policy	TLSv1.2_2021



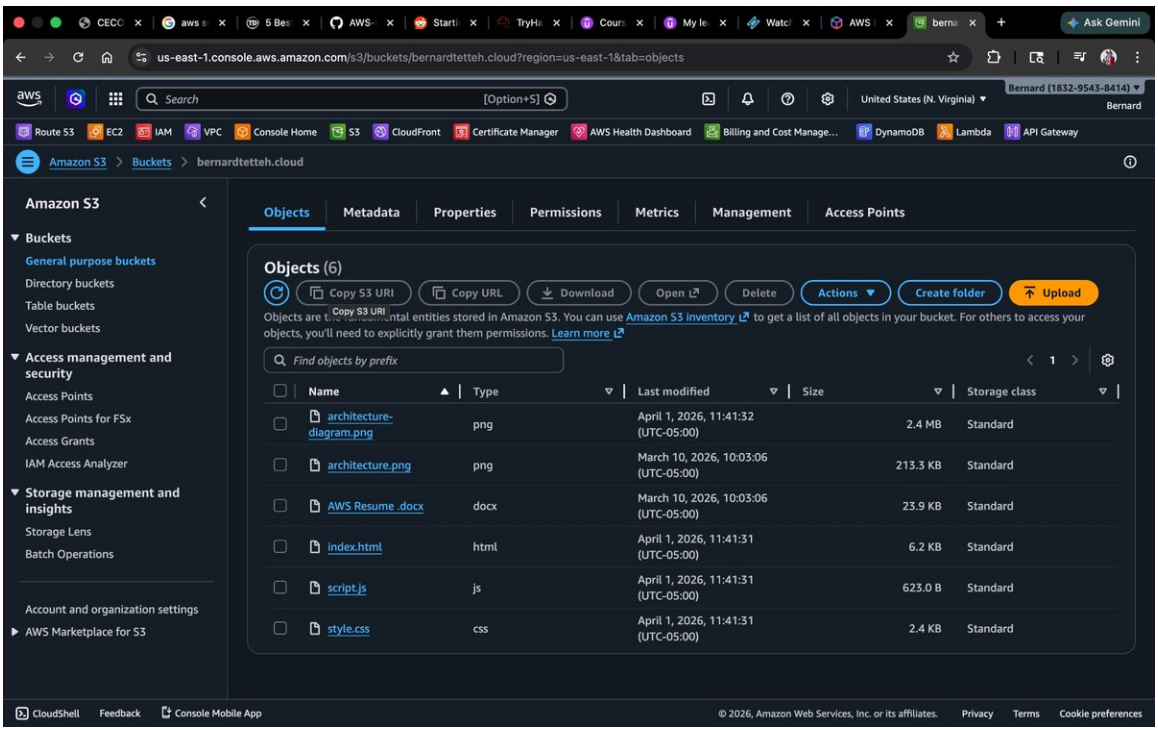
CloudFront distribution bernardtetteh-cloud-resume. Custom domain and ACM SSL certificate attached.

04 Amazon S3 — Static Website Hosting

Amazon S3 stores all the website files. CloudFront uses S3 as its origin when a visitor requests a file not cached at the nearest edge, CloudFront fetches it from S3. No servers involved just files served from storage, making it extremely cheap and reliable.

→ index.html — main resume page (6.2 KB)
→ script.js — visitor counter JavaScript (623 bytes)
→ style.css — all styling (2.4 KB)
→ architecture-diagram.png — architecture diagram on the site (2.4 MB)

Setting	Value
Bucket name	bernardtetteh.cloud
Static website	Enabled
Objects uploaded	6 files
Access	Public read via bucket policy



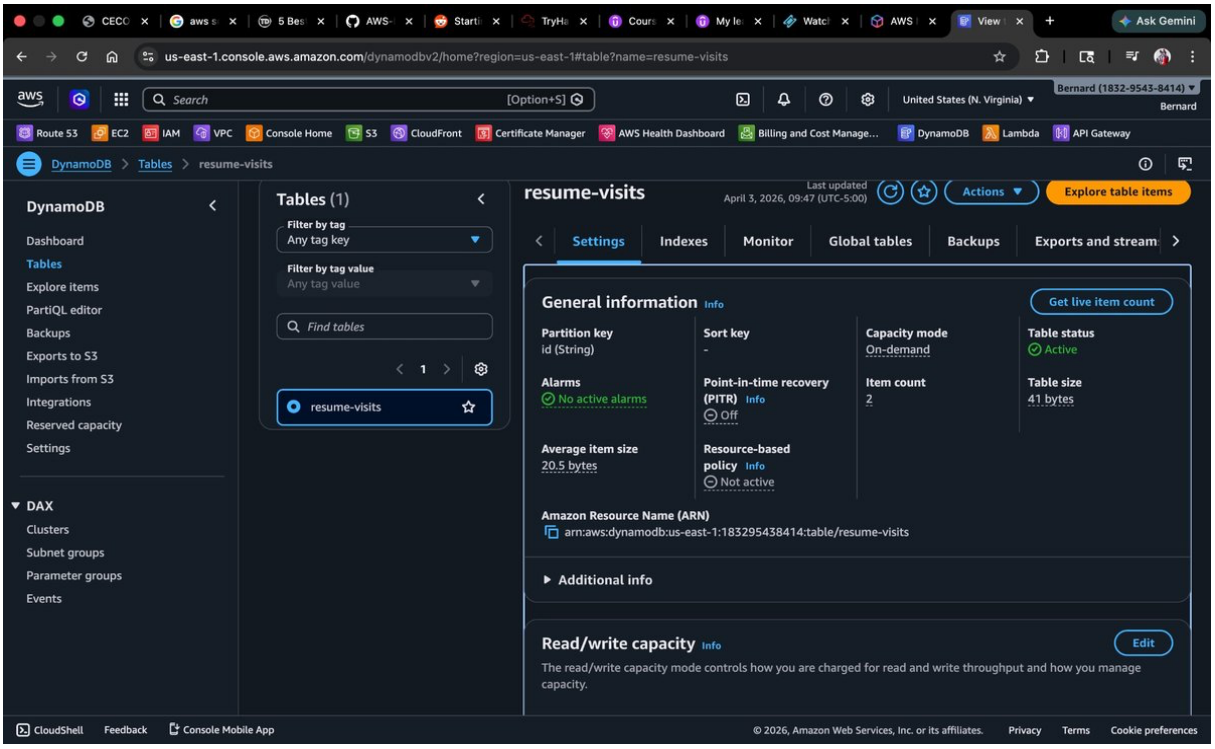
S3 bucket bernardtetteh.cloud — all 6 uploaded objects including index.html, script.js, style.css.

05 Amazon DynamoDB

DynamoDB is Amazon's NoSQL database that permanently stores the visitor count. Even if the website or Lambda function is rebuilt, the count survives. OnDemand capacity means I only pay when the database is actually accessed.

→ Table name: resume-visits
→ Partition key: id (String)
→ Row structure: id = "counter", visits = N (increments each visit)
→ Capacity mode: On-demand — auto-scales, pay per request only
→ Status: Active

Setting	Value
Table name	resume-visits
Partition key	id (String)
Data stored	id = "counter", visits = N
Capacity	On-demand
Status	Active



DynamoDB table resume-visits — on-demand capacity, Active status, partition key: id (String).

06 AWS Lambda

Lambda is the serverless compute service running the visitor counter logic. No servers to manage. Lambda activates only when API Gateway triggers it, runs the Python code in under a second, then shuts down. Zero cost when idle, auto scales under any load.

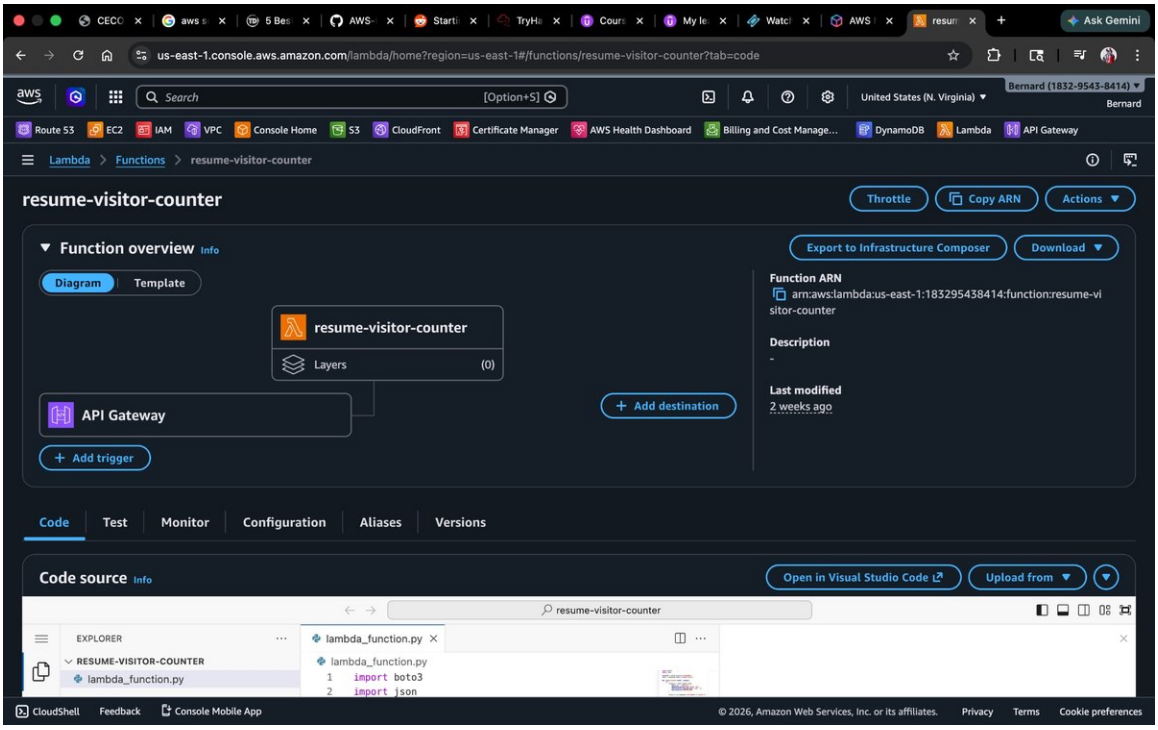
→ Receives trigger from API Gateway when a visitor loads the page
→ Connects to DynamoDB table resume-visits
→ Atomically adds 1 to visits — safe under concurrent requests
→ Returns the new count as JSON: {"visits": 42}
→ Includes CORS headers so the browser accepts the response

Setting	Value
Function name	resume-visitor-counter
Runtime	Python 3.12
Trigger	API Gateway (HTTP API)
IAM permission	AmazonDynamoDBFullAccess
Package size	511 bytes

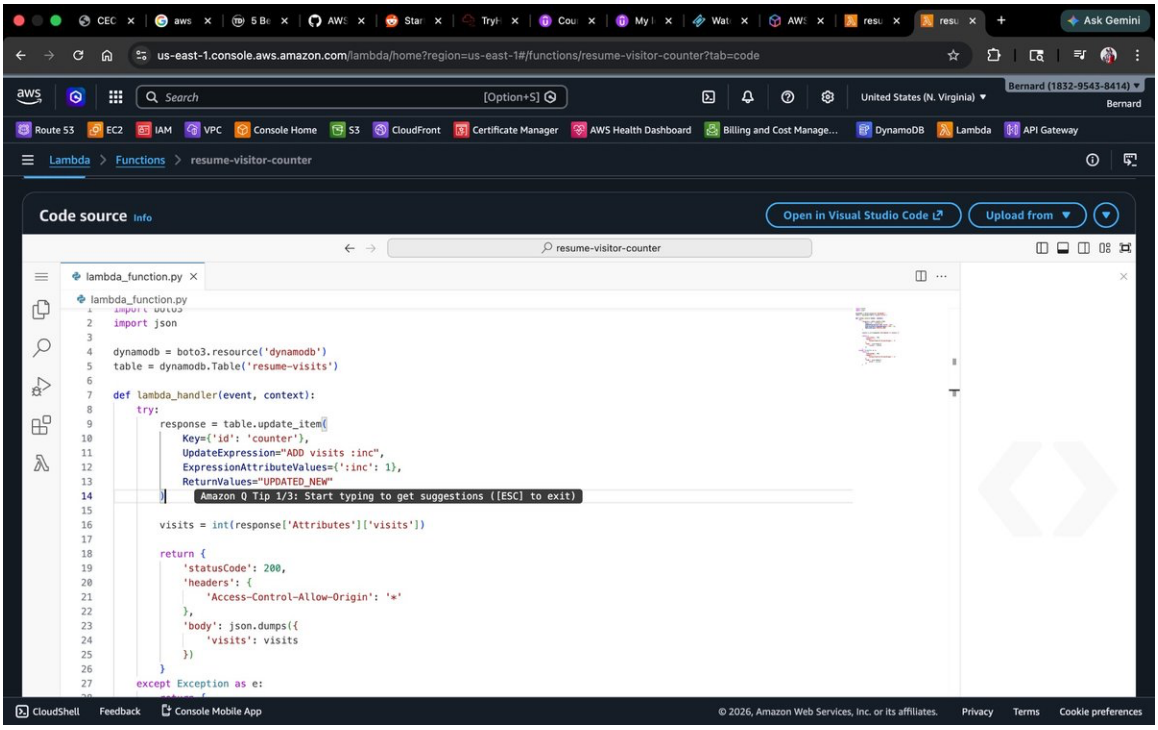
My Lambda Code for the project.

```
import boto3
import json

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('resume-visits')

def lambda_handler(event, context):
    try:
        response = table.update_item(
            Key={'id': 'counter'},
            UpdateExpression="ADD visits :inc",
            ExpressionAttributeValues={':inc': 1},
            ReturnValues="UPDATED_NEW"
        )
```

```
        visits = int(response['Attributes']['visits'])

        return {
            'statusCode': 200,
            'headers': {
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps({
                'visits': visits
            })
        }
    except Exception as e:
        return {
            'statusCode': 500,
            'headers': {
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps({
                'error': str(e)
            })
        }
```



Lambda function overview — resume-visitor-counter with API Gateway shown as the trigger source.



Lambda function code deployed in AWS console — the complete Python function with try/except error handling.

07 Amazon API Gateway

API Gateway is the front door to the backend. When the website JavaScript runs fetch(API_URL), that request hits API Gateway first. It receives the GET /count request and forwards it to Lambda. It also handles CORS — the browser security policy that controls which websites can receive responses from the API.

→ Type: HTTP API — simpler and cheaper than REST API
→ Route: GET /count — the endpoint the browser calls
→ Integration: Lambda function resume-visitor-counter
→ CORS enabled — required so the browser accepts the response
→ Auto-deploy on — changes go live immediately

Setting	Value
API type	HTTP API
Route	GET /count
Integration	Lambda (resume-visitor-counter)
CORS	Enabled — Access-Control-Allow-Origin: *
Deploy	Auto-deploy on

REAL PROBLEMS. REAL FIXES.

Challenges Faced & How I Solved Them

Building this project was not smooth. Every challenge below is a real problem I hit during the build and solving each one deepened my understanding of AWS far more than any tutorial could.

#1 S3 Static Website — 404 Errors

■ CHALLENGE	✓ SOLUTION
After uploading files to S3 and enabling static website hosting, visiting the URL returned a 404 error. The bucket appeared correctly configured but files were not loading.	The issue was a combination of three things: files were uploaded inside a subfolder instead of the root, the bucket policy was missing the public read permission, and the index document setting was not saved correctly.

■ EXACT FIX

1. The verified files (index.html, script.js, style.css) were at the root of the bucket not inside a folder.
2. I applied the correct bucket policy granting s3:GetObject to Principal: "*" on the bucket ARN.
3. I went to Properties → Static website hosting → confirmed index document set to index.html.
4. I tested the S3 website endpoint directly before wiring CloudFront.

#2 API Integration Failure — CORS Error

■ CHALLENGE	✓ SOLUTION
The API Gateway endpoint worked perfectly when tested directly in the browser or Postman. But when the website JavaScript	CORS is a browser security rule that blocks responses from a different domain unless the server explicitly allows it. The fix required enabling

called it using fetch(), the browser blocked the response with a CORS policy error.	CORS in two places: API Gateway settings AND the Lambda response headers.

■ EXACT FIX

1. In API Gateway → selected the route → enabled CORS → set Access-Control-Allow-Origin to *
2. Added the header in Lambda: "Access-Control-Allow-Origin": "*" inside every return statement
3. Redeployed the Lambda function after the header change
4. Confirmed the fix by checking browser DevTools → Network tab → response headers

#3 DynamoDB Decimal Serialization Error

■ CHALLENGE	✓ SOLUTION
The Lambda function crashed with a JSON serialization error. DynamoDB was returning the visitor count as a Decimal type a Python data type that the json.dumps() function cannot convert automatically.	DynamoDB uses the Decimal type internally for all numbers to preserve precision. The fix was a one line change wrap the value in int() before passing it to json.dumps().

■ EXACT FIX

1. I identified the error: "Object of type Decimal is not JSON serializable" in Lambda logs
2. I located the line: visits = response["Attributes"]["visits"]
3. I processed to changed it to: visits = int(response["Attributes"]["visits"])
4. I redeployed and re-tested serialization error resolved

#4 API Endpoint URL & Route Config Errors

■ CHALLENGE	✓ SOLUTION
The visitor counter was returning DNS errors and failed requests. The endpoint URL being used in the JavaScript was either incorrect, pointing to the wrong stage, or the /count route had not been deployed properly.	Traced the problem step by step: verified the correct Invoke URL from the API Gateway console, confirmed the /count route existed and was integrated with Lambda, then checked that auto-deploy was enabled.

■ EXACT FIX

1. I went to API Gateway → selected the HTTP API → copied the exact Invoke URL from the console.
2. Confirmed the route was GET /count not GET /views or GET / or anything else.
3. Checked the Lambda integration was attached to the correct function.
4. Enabled auto-deploy on the $default stage so route changes deployed immediately.
5. Updated the API_URL constant in script.js with the correct full endpoint URL.

#5 Visitor Counter Stuck on "Loading..."

■ CHALLENGE	✓ SOLUTION
The live visitor counter was permanently showing "Loading..." and never updating. The counter element existed in the HTML, but the number was never populated — even after the API was confirmed working.	Two issues were causing this. First, JavaScript was reading data.visits but Lambda was returning the key as views — a name mismatch. Second, the script.js file was not correctly linked in index.html.

■ EXACT FIX

1. Opened Visual studio code → Console confirmed fetch() was succeeding but data.visits was undefined
2. Checked Lambda return body: it returned {"views": N} not {"visits": N} key mismatch.
3. Updated JavaScript to read data.views to match the Lambda response key.
4. Verified script.js was linked correctly in index.html.
5. Re-uploaded both files to S3 and created a CloudFront invalidation to clear the cache.

#6 Website Not Updating After S3 Changes

■ CHALLENGE	✓ SOLUTION
After uploading updated files to S3, the live website kept showing the old version. Clearing the browser cache made no difference.	CloudFront caches files at edge locations worldwide until the TTL expires which can take 24+ hours by default. A CloudFront Invalidation (/*) forces all edge locations to fetch fresh files from S3 immediately.

■ EXACT FIX

1. I went to AWS Console → CloudFront → selected the bernardtetteh-cloud-resume distribution
2. Clicked the Invalidations tab → Create invalidation.
3. Entered /* in the object paths field and it invalidates every cached file.
4. Clicked Create invalidation and waited between ~30 seconds to 2 minutes for it to complete.
5. Refreshed the live site and changes appeared immediately.

FINANCIALS

Cost Breakdown

Serverless means you only pay for what you use. All seven services combined cost under $1 a month to run.

Service	Free Tier	Monthly Cost
Route 53	No free tier on hosted zones	**~$0.50**
CloudFront	1 TB + 10M requests/month	**$0.00**
ACM	Always free	**$0.00**
S3	5 GB + 20K requests/month	**$0.00**
API Gateway	1M requests/month	**$0.00**
Lambda	1M requests/month	**$0.00**
DynamoDB	25 GB + 25 WCU/RCU free	**$0.00**
TOTAL		**~$0.50/month**

TAKEAWAYS

What I Learned

→ DNS: How Route 53 translates domain names to IP addresses using A records and Alias records
→ CDN: How CloudFront caches content globally and why CloudFront invalidations are needed after updates
→ TLS: How SSL certificates work, why ACM must be in us-east-1, and how DNS validation works

→ Storage: How S3 static website hosting works and how to configure bucket policies correctly
→ Serverless: Why Lambda costs less than servers and how it only runs when triggered
→ Databases: How DynamoDB stores data, why Decimal types must be converted, and how atomic ADD works
→ CORS: What CORS is, why browsers enforce it, and that it must be fixed in BOTH API Gateway and Lambda
→ IAM: How roles and policies control which AWS services can communicate with each other
→ Debugging: How to use browser DevTools, Lambda logs, and API Gateway test tools to trace errors

---

Bernard Tetteh · AWS Certified Solutions Architect · bernardtetteh.cloud · www.linkedin.com/in/bernardtettehb49t · github.com/tele880